

PROJETO FINAL

1. Objetivo

Evoluir o projeto do modulo anterior da API Spring Boot adicionando uma camada de **Nginx**, implementando mecanismos de:

- Segurança
- Controle de tráfego
- Performance
- Organização arquitetural

O objetivo é transformar a API em uma aplicação mais robusta, próxima de um ambiente produtivo real.

2. Cenários Permitidos

Existem dois cenários possíveis:

Cenário A — Com Docker

- Nginx deve rodar em container.
- API Spring Boot deve rodar em container.
- Comunicação via rede Docker.

Cenário B — Sem Docker

- Nginx instalado localmente.
- API rodando via `java -jar` ou IDE.
- Comunicação via `localhost`.

OBS: O aluno **não precisa migrar para Docker** se ainda não estiver utilizando.

3. Requisitos Obrigatórios

3.1 Reverse Proxy

O Nginx deve:

- Escutar na porta 80
- Encaminhar requisições para a API Spring Boot
- A API não deve mais ser acessada diretamente pela porta original (ex: 8080)

Teste obrigatório:

- A API deve funcionar via `http://localhost`
- Se acessar a porta direta da API (ex: 8080), deve ficar claro que produção usaria apenas o proxy

3.2 Headers de Segurança

O Nginx deve adicionar os seguintes headers:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `X-XSS-Protection: 1; mode=block`

Deve ser possível visualizar esses headers via navegador ou Postman.

3.3 Rate Limiting

Implementar limitação de requisições por IP:

- Máximo: 5 requisições por segundo
- Burst permitido: 10

Teste obrigatório:

- Disparar múltiplas requisições rápidas
- Comprovar retorno HTTP 429

3.4 Limite de Payload

Configurar limite máximo de upload:

- Máximo de 1MB por requisição

Teste obrigatório:

- Enviar payload maior que 1MB
- Comprovar erro HTTP 413

3.5 Compressão

Ativar compressão GZIP no Nginx para:

- `application/json`
- `text/plain`

Comprovar via header:

- `Content-Encoding: gzip`

3.6 Log Estruturado

Nginx deve registrar:

- IP do cliente
- Método HTTP
- Status
- Tempo de resposta do upstream

Apresentar exemplo real do log funcionando.

4. Requisitos Intermediários

Escolher pelo menos **2 dos itens abaixo**:

4.1 Cache de GET

- Cachear respostas GET por 10 segundos
- Não cachear requisições autenticadas

4.2 Basic Auth no Swagger

- Proteger `/swagger` ou `/actuator` com autenticação básica via Nginx

4.3 Redirecionamento HTTP → HTTPS (simulado)

- Criar configuração preparada para HTTPS (mesmo que seja autoassinado)

4.4 Custom Error Pages

- Página personalizada para 404
- Página personalizada para 50x

5. Requisitos Opcionais

- Implementar balanceamento entre 2 instâncias da API
- Implementar microcache (1–3 segundos)
- Implementar geração de `X-Request-ID`
- Integrar logs da aplicação com correlação de `request-id`
- Configurar health check com `/actuator/health`

6. Entregáveis

Todos devem entregar:

1. Repositório Git organizado
2. Arquivo `nginx.conf` comentado
3. README contendo:
 - Arquitetura utilizada
 - Como executar
 - Como testar cada requisito
 - Evidências (prints ou comandos utilizados)

7. Restrições

- Não é permitido alterar o código da API apenas para “forçar” funcionamento.
- O Nginx deve resolver os problemas na camada de infraestrutura.
- Configurações devem ser explícitas (não usar defaults implícitos).

8. Roteiro Obrigatório de Apresentação (10 minutos)

A apresentação deve seguir exatamente esta estrutura:

Minuto 0–1: Contextualização

- Qual é o projeto?
- Qual problema de robustez ele tinha antes?

Minuto 1–3: Arquitetura

Mostrar:

- Diagrama simples:
Cliente → Nginx → Spring Boot
- Explicar portas
- Explicar se está usando Docker ou não

Minuto 3–5: Demonstração Técnica

Demonstrar ao vivo:

- API funcionando via Nginx
- Headers de segurança visíveis
- Rate limiting gerando 429
- Payload maior que 1MB sendo bloqueado

Minuto 5–7: Configuração

Mostrar trechos do `nginx.conf` explicando:

- Bloco `server`
- `location`
- `Rate limit`
- `Gzip`
- `Proxy pass`

Minuto 7–9: Melhorias Extras

Explicar:

- O que foi implementado além do obrigatório
- Por que isso melhora robustez
- Impacto em ambiente real

Minuto 9–10: Conclusão

Responder:

- O que mudaria para produção real?
- Quais problemas ainda existem?
- O que aprendeu tecnicamente?